SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5 **Total Marks:50**

Annexure A

SIMULATION TASK

Code of Conduct:

I _Juhi Surin (192525148)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Juhi Surin

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools

Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Annexure A

SIMULATION TASK

QUESTIONS

1.Given the three-address code sequence:
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
Simulate the **peephole optimization** process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied, including algebraic simplification, constant folding, and dead code elimination.

2.Partition the following intermediate code into **basic blocks** and construct the corresponding **control flow graph**. Identify all leader statements and mention the rules used to identify them:
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2

---

3. Construct the **DAG** for the following basic block:
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
Using the DAG, identify all **common sub-expressions**. Then simulate a simple code generator to produce optimized target code and explain the register allocation decisions at each step.

---

4. For a target machine with two registers **R0 and R1**, simulate the working of a **simple code generator** for the code:
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
Show the **register descriptor** and **address descriptor** after each instruction is processed.

---

5. For a **RISC-style target machine** with instructions **LOAD, STORE, ADD, SUB, MUL, MOV**, simulate target code generation for the expression:
(a - b) * (c + d) - e
Show the complete sequence of target instructions generated. Also explain the **runtime storage management** and **stack frame layout** used during the computation.
Given the three-address code sequence:
t1 = a + b,
t2 = t1 + 0,
t3 = t2 * 1,
if t3 == 0 goto L1,
t4 = t3 + t3

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1

SIMULATION TASK — ANSWER SCRIPT

Student Name: Juhi Surin

Register Number: 1925252418

Course Code: CSA1407

Course Name: Compiler Design

Course Outcome: CO5 — AT1 (Simulation Task)

Faculty In-Charge: Dr. N. Deepa

Date of Submission: 26-08-2026

SIMULATION TASK: CODE OPTIMIZATION & CODE GENERATION

Question 1: Peephole Optimization

Given three-address code:

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

1.1 Conceptual Understanding

Peephole optimization examines a small "window" (peephole) of a few consecutive instructions and replaces them with an equivalent but cheaper sequence. It relies on local patterns: algebraic identities (x*1=x, x-0=x, x*0=0), constant folding (evaluating constant expressions at compile time), strength reduction (replacing an expensive operator with a cheaper one), and elimination of unreachable / dead code that can never affect the program's observable behaviour.

1.2 Model Design — Simulation of the Peephole Pass

The peephole window is slid over the instruction sequence and each statement is examined against known optimization rules, one pass at a time, as shown below.

Step	Instruction	Pattern Recognized	Optimization Applied	Result
1	t1 = x * 1	Multiplication by 1	Algebraic simplification	t1 = x
2	t2 = t1 + y	t1 already = x	Copy propagation	t2 = x + y
3	t3 = t2 - 0	Subtraction of 0	Algebraic simplification	t3 = t2 (= x + y)
4	t4 = t3 * 0	Multiplication by 0	Constant folding	t4 = 0
5	if t4 != 0 goto L1	t4 is the constant 0	Constant propagation → condition always FALSE	branch never taken
6	(branch of step 5)	Unreachable jump to L1	Dead-code elimination	statement removed
7	t5 = y + y	Addition of identical operands	Strength reduction	t5 = y << 1 (2*y)

1.3 Tool / Technique Used

A manual compiler-construction technique equivalent to the peephole optimizer module of a compiler back end was simulated: each instruction was matched against a rule table (algebraic identities → constant folding → copy propagation → dead-code elimination → strength reduction), applied iteratively until no further rule fires (a fixed point is reached).

1.4 Optimized Output Code

```
t2 = x + y
t5 = y << 1      // equivalent to t5 = 2 * y
```

1.5 Analysis of Results

The original 6 statements are reduced to 2. Because t4 always evaluates to the constant 0, the conditional branch is provably never taken, so the entire branch and the dead label reference are safely removed without changing program semantics. The chain t1→t2→t3 collapses through copy propagation into a single expression x + y, and the redundant addition y + y is converted to a single left-shift, which executes faster than a general addition on most target machines. This demonstrates how local, low-cost transformations can produce a globally simpler and faster instruction stream.

Question 2: Basic Blocks and Control Flow Graph (CFG)

```
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2
```

2.1 Conceptual Understanding

A basic block is a maximal sequence of consecutive statements with a single entry point (the leader) and single exit point, such that control enters only at the leader and leaves only at the last statement, with no branching in-between. The control flow graph (CFG) is built with basic blocks as nodes and possible flows of control (fall-through or jump) as directed edges.

2.2 Rules Used to Identify Leaders

- Rule 1: The first statement of the program is a leader → statement (1).
- Rule 2: The target of any conditional or unconditional goto is a leader → statement (6) is the target of "goto 6", statement (7) is the target of "goto 7".
- Rule 3: Any statement that immediately follows a conditional or unconditional goto is a leader → statement (4) follows the conditional at (3).

Leaders identified: statements (1), (4), (6), (7).

2.3 Model Design — Basic Block Partition

Block	Statements	Leader	Exit Behaviour
B1	(1) x=a+b (2) y=x*c (3) if y>50 goto 6	(1)	Conditional branch → B3 (true) or fall-through → B2 (false)
B2	(4) z=y-d (5) goto 7	(4)	Unconditional jump → B4
B3	(6) z=y+d	(6)	Fall-through → B4
B4	(7) w=z*2	(7)	Program exit / return

2.4 Tool / Technique Used

The standard two-step algorithm for CFG construction was simulated manually: (i) apply the three leader-identification rules to mark block boundaries, (ii) place each leader and all statements up to (but not including) the next leader into one block, then (iii) connect blocks with directed edges representing every possible transfer of control (fall-through and jump).

2.5 Control Flow Graph (Edge List)

From	To	Condition
B1	B3	y > 50 is TRUE (branch taken)
B1	B2	y > 50 is FALSE (fall-through)
B2	B4	unconditional goto 7
B3	B4	fall-through

2.6 Analysis of Results

The CFG shows a classic if-then-else shape: B1 is the predicate block with two successors (B2, B3), and both converge at the common successor B4, which makes B4 an ideal point for inserting phi-like merge information in later data-flow passes (e.g., for computing z's reaching definitions before w=z*2 executes). Because B2 and B3 both define z independently before B4 uses it, this structure is precisely the kind of join point where reaching-definitions analysis and later optimizations (such as common sub-expression elimination across blocks) must be applied with care.

Question 3: DAG Construction and Code Generation

```
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
```

3.1 Conceptual Understanding

A Directed Acyclic Graph (DAG) represents the expressions of a basic block with interior nodes as operators and leaf nodes as operands (identifiers/constants). Whenever the same computation appears more than once, the DAG reuses a single node for it — this is exactly how common sub-expressions are detected automatically during DAG construction, rather than needing a separate comparison pass.

3.2 Model Design — DAG Structure

Node	Expression	Represents	Notes
N1	a + b	p, q, s	Built once; p, q and s are attached as extra labels on the same node since a+b is computed identically each time
N2	N1 * N1	r	r = p * q = (a+b) * (a+b), uses N1 twice
N3	N2 + N1	t	t = r + s reuses N1 again for s

3.3 Common Sub-expressions Identified

p, q and s are all textually identical (a + b) and therefore collapse to the single DAG node N1. Without the DAG, a naive compiler would compute a+b three separate times; the DAG exposes that only one computation of a+b is actually required.

3.4 Tool / Technique Used — Simple Code Generator with Register Allocation

The DAG was traversed bottom-up (leaves to root) and target code was generated node by node using a 2-register machine, applying the standard getReg() heuristic: reuse a register already holding an operand when possible, and allocate a fresh register only when a value must survive past the point where another register is reused.

Step	Target Instruction	Meaning	R0	R1
1	MOV R0, a	R0 = a	a	–
2	ADD R0, b	R0 = a+b (= N1: p,q,s)	N1	–
3	MOV R1, R0	save a copy of N1 (needed again as s)	N1	N1
4	MUL R0, R0	R0 = N1*N1 = r	r	N1 (=s)
5	ADD R0, R1	R0 = r + s = t	t	– (freed)
6	MOV t, R0	store result to memory location t	t	–

3.5 Analysis of Results

Only one MOV/ADD pair computes a+b (step 1-2) instead of three, directly saving 4 redundant instructions that a non-DAG-based generator would otherwise emit for q and s. A second register (R1) is essential: after N1 is destroyed by the MUL at step 4 (since R0 is overwritten), the earlier saved copy in R1 supplies the value of s for the final addition. This illustrates the trade-off central to register allocation — saving a live value costs one extra register/instruction

now, but avoids recomputation later; the DAG is precisely what tells the code generator which values are still needed later (i.e., have more than one parent) and must therefore be kept alive.

Question 4: Simple Code Generator with Register & Address Descriptors

Target machine: 2 registers R0, R1.

```
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
```

4.1 Conceptual Understanding

A register descriptor keeps track of what value each register currently holds; an address descriptor keeps track of where the current value of a name can be found at run time (register, memory, or both). The simple code generator algorithm (Aho–Ullman) chooses target registers using a getReg() function based on these two descriptors so that live values are not overwritten before use, while free registers are reused as soon as a value dies.

4.2 Model Design / Tool Usage — Instruction-by-Instruction Simulation

#	TAC	Target Code Generated	Register Descriptor (after)	Address Descriptor (after)
1	t1 = a + b	MOV R0, a ADD R0, b	R0 = t1	t1 → R0
2	t2 = c * d	MOV R1, c MUL R1, d	R0 = t1, R1 = t2	t1 → R0, t2 → R1
3	t3 = t1 - t2	SUB R1, R0 (R0 = R0 − R1)	R0 = t3, R1 = free (t2 dead)	t3 → R0
4	t4 = e + f	MOV R1, e ADD R1, f	R0 = t3, R1 = t4	t3 → R0, t4 → R1
5	t5 = t3 / t4	DIV R1, R0 (R0 = R0 ÷ R1)	R0 = t5, R1 = free (t4 dead)	t5 → R0
6	store t5	MOV t5, R0	R0 = t5	t5 → R0 and memory

Note: SUB R1,R0 is written to mean "subtract the contents of R1 from R0, result in R0", matching the two-operand form typical of RISC/CISC ALU instructions; the same convention is used for DIV.

4.3 Analysis of Results

With careful ordering, both temporaries (t1, t2) are computed before either is needed, and each is retired (freed) exactly at the instruction that consumes it (t3, then t5), so only 2 registers are ever required for the entire five-statement sequence — no spilling to memory is necessary. This is possible because the expression tree (t1−t2) and (t3÷t4) has at most two live values at any point, which matches exactly the number of available registers; had a third value needed to stay live simultaneously, the generator would have been forced to spill one temporary to memory.

Question 5: RISC Target Code Generation and Runtime Storage

Instruction set: LOAD, STORE, ADD, SUB, MUL, MOV. Expression: (a − b) * (c + d) − e

5.1 Conceptual Understanding

On a load/store (RISC) architecture, the ALU can only operate on values already in registers, so every operand must first be explicitly LOADed from memory; results are written back with STORE only when they must persist beyond the current computation. The expression is first reduced to three-address code, then each TAC statement is mapped to a small sequence of LOAD/ALU/STORE instructions.

5.2 Model Design — Three-Address Code

```
t1 = a - b
t2 = c + d
t3 = t1 * t2
t4 = t3 - e
```

5.3 Tool / Technique Used — Target Code Simulation (3 registers R0–R2)

Step	Instruction	Effect
1	LOAD R0, a	R0 = a
2	LOAD R1, b	R1 = b
3	SUB R0, R0, R1	R0 = a − b = t1
4	LOAD R1, c	R1 = c
5	LOAD R2, d	R2 = d
6	ADD R1, R1, R2	R1 = c + d = t2
7	MUL R0, R0, R1	R0 = t1 * t2 = t3
8	LOAD R1, e	R1 = e
9	SUB R0, R0, R1	R0 = t3 − e = t4 (final result)
10	STORE result, R0	write t4 to its home memory location

5.4 Runtime Storage Management and Stack Frame Layout

If this expression is evaluated inside a procedure, its operands (a, b, c, d, e) and the result live in that procedure's activation record on the runtime stack, organised (from higher to lower addresses, growing downward) as follows:

Region (high → low address)	Purpose
Actual parameters	Values/addresses passed by the caller
Return address	Address in the caller to resume execution after the call returns
Control (dynamic) link	Pointer to the caller's activation record, used to restore the caller's frame on return

Region (high → low address)	Purpose
Access (static) link / saved registers	Supports nested scoping and preserves callee-saved registers such as R0–R2 used above
Local variables (a, b, c, d, e if local)	Storage for the procedure's own declared variables
Temporaries (t1–t4)	Space reserved for compiler-generated temporaries when registers are insufficient (spilling)

The stack pointer (SP) marks the current top of the stack and the frame pointer (FP) gives stable offsets into the current activation record even as SP moves during nested expression evaluation. During this computation, t1–t4 were kept entirely in registers (R0–R2) and never spilled to the temporaries area, because the maximum number of simultaneously live values (2) did not exceed the number of available registers (3).

5.5 Analysis of Results

The generated code uses exactly 10 instructions and never spills to the stack, which is optimal for this expression tree given three available registers: each subtree (a−b) and (c+d) is fully evaluated and consumed before the next value is needed, so no temporary must outlive the register that holds it. This demonstrates the general principle that RISC code generation quality depends on evaluation order — evaluating the more register-hungry subtree first (here, either side works since both need exactly 2 registers) avoids unnecessary spills to the activation record's temporaries area.

Overall Summary

Across all five simulations, the same underlying compiler back-end pipeline was exercised end-to-end: peephole-level local optimization (Q1), control-flow structuring via basic blocks and CFG (Q2), redundancy elimination via DAG-based common sub-expression detection (Q3), and register-aware target code generation with explicit register/address descriptors on both an accumulator-style (Q4) and a load/store RISC-style (Q5) machine. Together these confirm CO5: the ability to develop code optimization and code generation using DAG representation with data-flow information to produce correct, efficient target code.